paytag

Shopping Made Effortless

Developer Skill Test

1. Introduction

Hi, and thank you for your interest in joining our team!

This task is an opportunity for you to showcase your habits of work and your way of designing a software solution.

The process goes as follows:

1. After you submit, we'll schedule a review session where you walk us through your code and the design choices you made.

2. Summary

Your first task as a **PayTag Developer**, will be implementing a fast-checkout client to our self-checkout machines. The machine scans the **RFID** tags on items dropped into it, the POS rings the basket up, and once payment is confirmed the machine **neutralizes** the security tags so the items can pass through the gate.

In production, your code would run as part of the POS integration, talking to the machine over the store's local network. For this exercise you'll be running against a **PayTag simulator** — no production rig required.

2.1 Composition of the System

- **PayTag Machine (simulated):** A self-checkout terminal exposing a JSON HTTP REST API. Hosts an RFID reader and a neutralizer. For this exercise the machine is a simulator process running on your machine.
- **API:** Three endpoints — a health check, an item-scan call, and a neutralize call. JSON in both directions.
- **Persistence:** A local **MongoDB** instance your client writes to (install natively or run in Docker — your choice).

2.2 What's Already Set Up

- A PayTag simulator is provided with this brief; get it running on your machine so it's reachable at `localhost:8765`.
- The API endpoints are stable and documented in the API Reference below. JSON request and response shapes are fixed; error codes are a documented numeric enum.

- A Postman collection (`Task_Postman_Collection.json`) is provided with `host` and `port` pre-filled. Each endpoint has a working example request and example responses for both success and error paths.
- Tagged items are baked into the simulator, including at least one `IsHardTag` item so you can exercise the hard-tag requirement.

2.3 App Flow

1. The user presses the configured scan key (default **S**) anywhere on the system (e.g. while focused on a POS field in another app) — a checkout session **starts**. The operator loads tagged items onto the machine; your client should surface what is in the basket while the session is open.
2. Pressing the neutralize key (default **N**) **ends the session**: the basket is neutralized so the items can pass the gate, and the outcome — including anything that failed — is reported clearly.
3. Everything meaningful that happened — the session, its items, the neutralization outcome, and the technical record of the run — is persisted to **MongoDB**.
4. `Ctrl+C` exits the script cleanly.

3. Your Task

1. **Checkout sessions** — the scan key (default **S**), pressed anywhere on the system, starts a checkout session and shows the operator what is in the basket; the neutralize key (default **N**) closes the session and neutralizes the basket. The endpoints are documented below.
2. **MongoDB persistence** — persist the application's data to a local MongoDB. Design the architecture so that **business data** (transactions, scanned items, neutralization outcomes) and **technical / code-level data** (run logs, errors, machine health checks) live in **separate collections**. Which collections you create, what each document looks like, and where the line between business and technical falls is yours to design — document your schema design.
3. **Configuration via YAML** — no tunable value hardcoded; settings load from a YAML configuration file.
4. **Hard-tag awareness** — hard tags need physical removal, not electronic neutralization.
5. **Plan in writing before coding**: which endpoints you'll call, where the hotkey logic lives, your MongoDB collection design, how you'll cover each error path, what the terminal output will look like, and which alternatives you considered. Include this plan in your repository.

API Reference

1. `GET /info` — health check. Returns the machine's version and the status of each hardware component.

Response:

```
{
  "version": "3.8.34",
  "machine_status": {
    "reader_status": true,
    "neutralizer_status": true,
    "all_connected": true
  }
}
```

2. **`POST /partner/GetItems`** — read items currently on the antenna.

Request:

```
{
  "TransactionNumber": "P01112340"
}
```

Response:

```
{
  "TransactionNumber": "P01112340",
  "Items": [
    {
      "Barcode": "7290000000001",
      "RFID": "E200001B0000000000000001",
      "IsHardTag": false
    }
  ],
  "ErrorCode": 0,
  "ErrorMessage": null
}
```

3. **`POST /partner/Neutralize`** — deactivate the security tags on the items provided. Build the `Items` list from the `Barcode` and `RFID` values returned by `GetItems`.

Request:

```
{
  "TransactionNumber": "P01112340",
  "Items": [
    {
      "Barcode": "7290000000001",
      "RFID": "E200001B0000000000000001"
    }
  ]
}
```

Response:

```
{
  "TransactionNumber": "P01112340",
  "NeutralizedRFItems": [
    {
      "Barcode": "7290000000001",
      "RFID": "E200001B0000000000000001",
      "IsHardTag": false
    }
  ],
  "FailedRFItems": [],
  "ErrorCode": 0,
  "ErrorMessage": null
}
```

Error Codes

Code	Name	Meaning
0	None	Success
1	GeneralError	General error
4	ReaderNotConnected	RFID reader is disconnected
8	NeutralizerNotConnected	Neutralizer is disconnected
17	NotAllTagsNeutralized	Some items failed neutralization

Notes:

1. The code needs to be clean, structured, and easy to follow.
2. Feel free to use an AI assistant — but know that you will be accountable on the implementation, design choices and expected to show a deep understanding of the logic behind the app.

4. Material Provided

1. The **Postman collection** — `Task_Postman_Collection.json`, with host and port preconfigured.
2. The **PayTag simulator** — provided as `paytag-demo-api.tar.gz`; get it running on your machine so the API is reachable at `localhost:8765`. Tagged items are included.

5. Submission

1. Share a **git repository** (a link, or a zip including the `.git` directory) with your code, your written plan, your YAML configuration files, and a short README covering how to run it.